

Data Lakehouse

Guide d'installation et de finalisation du projet

Trino + Iceberg + ClickHouse + MinIO

*Ce guide vous accompagne dans toutes les étapes nécessaires pour faire fonctionner la pile complète.
Les deux interfaces utilisateur sont déjà construites. L'accent est mis sur l'infrastructure et l'initialisation des données.*

. Table des matieres

- 1 Vue d'ensemble du projet
- 2 Prerequis
- 3 Demarrage rapide - Docker Compose
- 4 Etape 1 : Corriger les permissions du bucket MinIO
- 5 Etape 2 : Initialiser ClickHouse avec des donnees exemple
- 6 Etape 3 : Initialiser les tables Iceberg via Trino
- 7 Etape 4 : Lancer les interfaces utilisateur
- 8 Reference des ports
- 9 Diagramme d'architecture
- 10 Depannage
- 11 Prochaines etapes et idees

1. Vue d'ensemble du projet

Il s'agit d'une plateforme data lakehouse moderne qui combine un lac de données (MinIO / S3) avec Apache Iceberg (pour les transactions ACID sur le lac), Trino (pour les requêtes SQL fédérées sur plusieurs sources) et ClickHouse (pour l'analytique columnaire rapide).

Le projet comporte deux applications frontales :

- data-product-ui -- Un tableau de bord d'analytique commerciale affichant le chiffre d'affaires, les unités vendues, les ventes par jour, les ventes par catégorie et les produits les plus performants. Interroge les vues ClickHouse dans la base ecommerce_product.
- query-explorer-ui -- Un explorateur de requêtes SQL fédérée avec un navigateur de catalogues/schemas/tables et un outil d'import CSV qui ingère les données dans ClickHouse via S3.

Les deux interfaces sont des applications React + Vite. Le code source, les dépendances (node_modules) et les builds de production (dist/) sont tous présents. Aucun codage frontal nécessaire.

Services

- MinIO (ports 9000, 9001) -- Stockage objet compatible S3. Bucket racine : warehouse
- PostgreSQL (port 5432) -- Stocke les métadonnées du catalogue Iceberg (base : iceberg_catalog)
- Trino (port 8080) -- Moteur de requêtes SQL distribué open-source
- ClickHouse (port 8123) -- Base de données analytique columnaire
- mc -- Conteneur d'initialisation qui crée le bucket warehouse sur MinIO au démarrage

2. Prerequis

Assurez-vous d'avoir installé les éléments suivants avant de commencer :

- Docker Desktop (ou Docker Engine + plugin Docker Compose)
- Node.js v18+ et npm (pour exécuter les interfaces hors de Docker)
- Un navigateur web (Chrome / Firefox / Edge)
- Au moins 8 Go de RAM allouée à Docker
- Git (optionnel, pour le contrôle de version)

Pour vérifier que Docker est prêt, exécutez :

```
docker compose version
docker info
```

3. Demarrage rapide - Docker Compose

Depuis le repertoire racine du projet, lancez tous les services avec :

```
docker compose up -d
```

Cela lance MinIO, PostgreSQL, Trino, ClickHouse et le conteneur d'initialisation mc.

Verifiez que tous les services fonctionnent :

```
docker compose ps
```

Attendez 20 a 30 secondes que les services s'initialisent (surtout Trino, qui prend du temps pour charger les connecteurs). Ensuite, verifiez les logs pour d'eventuelles erreurs :

```
docker compose logs --tail=50 trino
```

Remarque : Si un port est deja utilise, modifiez docker-compose.yml et changez le cote gauche du mapping de port (ex : '8081:8080' -> '8082:8080')

4. Etape 1 : Corriger les permissions du bucket MinIO

Le fichier docker-compose.yml actuel cree le bucket warehouse avec un acces public en lecture seule. Cependant, l'outil d'import CSV dans query-explorer-ui doit ecrire des fichiers dans MinIO. Vous devez mettre a jour la commande du conteneur d'initialisation mc.

Dans docker-compose.yml, trouvez le service mc et modifiez sa commande :

```
command:
  - /bin/sh
  - -c
  - |
    sleep 5
    mc alias set myminio http://minio:9000 minioadmin minioadmin
    mc mb myminio/warehouse --ignore-existing
    mc anonymous set public myminio/warehouse
    mc admin policy set myminio readwrite user=minioadmin
```

Le point essentiel : le bucket 'warehouse' doit autoriser l'écriture pour que l'import CSV fonctionne. La solution la plus simple est d'utiliser de

5. Etape 2 : Initialiser ClickHouse avec des donnees exemple

Le tableau de bord data-product-ui interroge trois vues ClickHouse dans la base de donnees ecommerce_product :

- ecommerce_product.daily_sales
- ecommerce_product.sales_by_category
- ecommerce_product.top_products

Ces tables/vues n'existent pas encore. Vous devez les creer. Le moyen le plus simple est de creer un fichier SQL d'initialisation que ClickHouse executera automatiquement au premier demarrage.

Marche a suivre :

1. Creez un dossier appele clickhouse-init a la racine du projet.
2. A l'interieur, creez un fichier nomme init.sql avec le contenu suivant :

```
CREATE DATABASE IF NOT EXISTS ecommerce_product;

USE ecommerce_product;

CREATE TABLE IF NOT EXISTS raw_sales (
    sale_date Date,
    product_name String,
    category String,
    quantity UInt32,
    unit_price Float64,
    total_amount Float64
) ENGINE = MergeTree()
ORDER BY (sale_date, category);

INSERT INTO raw_sales VALUES
('2025-01-01', 'Wireless Mouse', 'Electronics', 15, 29.99, 449.85),
('2025-01-01', 'Desk Lamp', 'Home & Office', 8, 45.00, 360.00),
('2025-01-02', 'USB-C Hub', 'Electronics', 22, 34.99, 769.78),
('2025-01-02', 'Notebook Set', 'Stationery', 30, 12.50, 375.00),
('2025-01-03', 'Mechanical Keyboard', 'Electronics', 10, 89.99, 899.90),
('2025-01-03', 'Standing Desk', 'Furniture', 3, 399.00, 1197.00),
('2025-01-04', 'Monitor Arm', 'Office', 12, 79.99, 959.88),
('2025-01-04', 'Coffee Mug', 'Kitchen', 50, 14.99, 749.50),
('2025-01-05', 'Webcam HD', 'Electronics', 18, 59.99, 1079.82),
('2025-01-05', 'Desk Organizer', 'Home & Office', 25, 22.99, 574.75);

CREATE MATERIALIZED VIEW IF NOT EXISTS daily_sales
ENGINE = SummingMergeTree()
ORDER BY sale_date
POPULATE AS
SELECT
    sale_date,
    SUM(quantity) AS units_sold,
    SUM(total_amount) AS revenue
FROM raw_sales
GROUP BY sale_date;

CREATE MATERIALIZED VIEW IF NOT EXISTS sales_by_category
```

```
ENGINE = SummingMergeTree()
ORDER BY category
POPULATE AS
SELECT
    category,
    SUM(quantity) AS units_sold,
    SUM(total_amount) AS revenue
FROM raw_sales
GROUP BY category;

CREATE MATERIALIZED VIEW IF NOT EXISTS top_products
ENGINE = SummingMergeTree()
ORDER BY total_revenue DESC
POPULATE AS
SELECT
    product_name,
    SUM(quantity) AS units_sold,
    SUM(total_amount) AS total_revenue
FROM raw_sales
GROUP BY product_name
ORDER BY total_revenue DESC;
```

3. Ajoutez un montage de volume pour ce dossier dans le service clickhouse dans docker-compose.yml :

```
services:
  clickhouse:
    ...
    volumes:
      - clickhouse_data:/var/lib/clickhouse
      - ./clickhouse-init:/docker-entrypoint-initdb.d # AJOUTEZ CETTE LIGNE
```

4. Redemarrez ClickHouse pour executer le script d'initialisation :

```
docker compose down clickhouse
docker compose up -d clickhouse
```

ClickHouse execute automatiquement les fichiers .sql dans /docker-entrypoint-initdb.d/ au premier démarrage. Si le volume clickhouse_data

6. Etape 3 : Initialiser les tables Iceberg via Trino

Le catalogue Iceberg est vide -- aucune table n'existe dans l'emplacement `s3://warehouse/`. Vous pouvez créer des tables Iceberg exemple en vous connectant à Trino et en exécutant des instructions DDL.

Se connecter à Trino

Utilisez la CLI Trino (dans Docker) ou un client JDBC :

```
# Option 1 : CLI Trino dans Docker
docker exec -it data-lakehouse-trino-1 trino

# Option 2 : Utilisez l'interface web sur http://localhost:8080/ui
```

Créer des tables Iceberg exemple

Une fois connecté à Trino, exécutez ces instructions SQL :

```
CREATE SCHEMA IF NOT EXISTS iceberg.lakehouse
WITH (location = 's3://warehouse/lakehouse');

CREATE TABLE IF NOT EXISTS iceberg.lakehouse.products (
  product_id BIGINT,
  name VARCHAR,
  category VARCHAR,
  price DOUBLE,
  created_at TIMESTAMP
) WITH (
  format = 'PARQUET',
  location = 's3://warehouse/lakehouse/products'
);

INSERT INTO iceberg.lakehouse.products VALUES
(1, 'Wireless Mouse', 'Electronics', 29.99, TIMESTAMP '2025-01-01 00:00:00'),
(2, 'Desk Lamp', 'Home & Office', 45.00, TIMESTAMP '2025-01-01 00:00:00'),
(3, 'USB-C Hub', 'Electronics', 34.99, TIMESTAMP '2025-01-02 00:00:00'),
(4, 'Mechanical Keyboard', 'Electronics', 89.99, TIMESTAMP '2025-01-03 00:00:00'),
(5, 'Standing Desk', 'Furniture', 399.00, TIMESTAMP '2025-01-03 00:00:00');

SELECT * FROM iceberg.lakehouse.products;
```

Après l'initialisation, vous pouvez explorer ces tables via `query-explorer-ui` sur `http://localhost:5173`.

7. Etape 4 : Lancer les interfaces utilisateur

Chaque interface est un serveur de dev Vite separe. Lancez-les dans deux terminaux.

Interface Data Product (Tableau de bord)

```
cd data-product-ui  
npm run dev
```

S'ouvre sur <http://localhost:5174>

Ce tableau de bord affiche les mesures de ventes depuis ClickHouse. ClickHouse doit etre en cours d'execution et initialise avec des donnees.

Interface Query Explorer

```
cd query-explorer-ui  
npm run dev
```

S'ouvre sur <http://localhost:5173>

Elle comporte deux onglets : un editeur de requetes SQL (connecte a Trino sur le port 8080) et un outil d'import CSV (televerse vers MinIO et importe dans ClickHouse).

Builds de production

Les deux applications ont deja des builds de production dans leurs dossiers dist/. Vous pouvez les servir avec n'importe quel serveur de fichiers statique (ex : nginx, serve).

```
# Option : servir le build de production  
npx serve data-product-ui/dist -l 4173  
npx serve query-explorer-ui/dist -l 4174
```

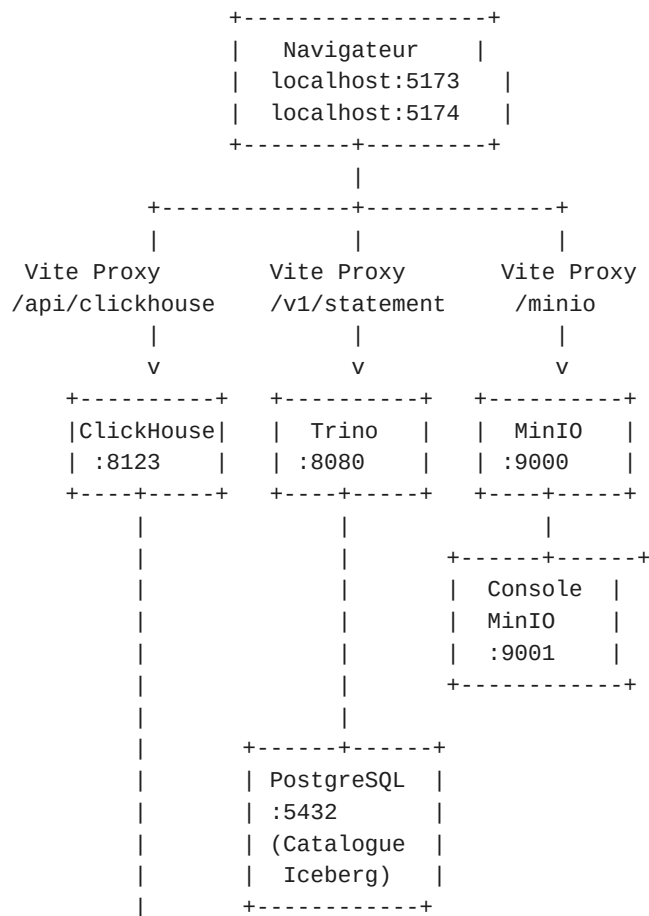
8. Reference des ports

Voici une reference rapide de tous les ports utilises par le projet :

5173	Query Explorer UI (serveur de dev)
5174	Data Product UI (serveur de dev)
8080	Trino (API REST et interface web sur /ui)
5432	PostgreSQL (catalogue Iceberg)
9000	API S3 MinIO
9001	Console MinIO (interface web)
8123	Interface HTTP ClickHouse

9. Diagramme d'architecture

Voici une representation textuelle de l'architecture du systeme :



10. Depannage

Les vues ClickHouse ne retournent aucune donnee

La base de donnees ecommerce_product et ses vues doivent etre creees. Suivez l'Etape 2 pour creer le script SQL d'initialisation et le monter dans le conteneur ClickHouse.

L'import MinIO echoue (403)

Le bucket warehouse a un acces public en lecture seule. Mettez a jour la commande mc dans docker-compose.yml pour autoriser l'ecriture. Voir Etape 1.

Trino n'affiche aucune table Iceberg

La base de donnees du catalogue PostgreSQL n'est peut-etre pas initialisee. Verifiez que postgres fonctionne et que Trino peut se connecter : docker compose logs trino | grep iceberg.

Port deja utilise

Si un port est deja pris, modifiez le cote gauche du mapping de port dans docker-compose.yml (ex : '8081:8080' -> '8082:8080').

npm run dev echoue

Assurez-vous que Node.js v18+ est installe. Supprimez node_modules et reinstallez :

```
cd data-product-ui && rm -rf node_modules && npm install
cd query-explorer-ui && rm -rf node_modules && npm install
```

Le conteneur Docker redemarre sans cesse

Verifiez les logs avec 'docker compose logs <nom-service>'. Causes courantes : dependances manquantes, mauvais identifiants ou conflits de ports.

Impossible de se connecter a Trino depuis query-explorer-ui

Assurez-vous que le proxy Vite dans query-explorer-ui/vite.config.js pointe vers la bonne URL Trino. La valeur par default est http://localhost:8080.

11. Prochaines etapes et idees

Une fois la configuration de base fonctionnelle, voici quelques idees pour prolonger le projet :

1. Ajouter plus de donnees exemple - Creez des jeux de donnees plus volumineux pour ClickHouse et Iceberg afin de rendre les tableaux de bord plus interessants.
2. Mettre en place dbt - Utilisez dbt avec l'adaptateur Trino pour gerer les transformations de donnees et les vues materialisees dans un flux de travail plus professionnel.
3. Ajouter Superset ou Grafana - Connectez Apache Superset ou Grafana a Trino ou ClickHouse pour des visualisations plus riches au-dela des interfaces integrees.
4. Automatiser avec un orchestrateur - Utilisez Apache Airflow ou Dagster pour planifier les pipelines d'ingestion et de transformation de donnees.
5. Ajouter une troisieme interface - Construisez une application simple Streamlit ou Dash qui interroge a la fois Iceberg (via Trino) et ClickHouse pour comparaison.
6. Containeriser les interfaces - Creez des Dockerfiles pour les deux applications React et ajoutez-les a docker-compose.yml pour que tout fonctionne avec une seule commande.
7. Ajouter CI/CD - Mettez en place des GitHub Actions pour lint, tester et builder les interfaces a chaque push.
8. Explorer les fonctionnalites Iceberg - Essayez les requetes de voyage dans le temps, l'evolution de schema et l'evolution de partition avec le connecteur Iceberg de Trino.

Bonne chance ! Si vous etes bloque, consultez les logs docker compose pour chaque service.

La racine du projet est : E:\New folder (25)\data-lakehouse